Basic Plots with Matplotlib

Code-along: data visualization with the GapMinder dataset

Marcus Birkenkrahe

COSC 1210 · CS I · Augustana University

Fall 2026

Objectives

- Data visualization using the GapMinder dataset
- Choosing the right plot (line, scatter, histogram)
- Customizing plots

Data visualization

Data visualization serves two purposes in data science:

Explore your data

The better you understand it, the better your insights.

Report your insights

A good plot lets the data tell its own story.

The choice of visualization depends on:

- The **data** you have
- The **story** you want to tell

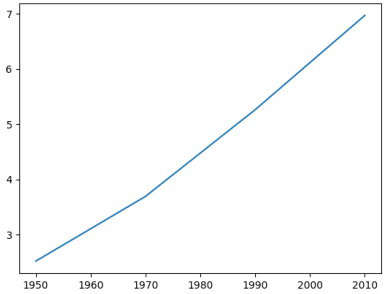
A first line plot

→ **Switch to editor · code-along · plot simple lists**

```
import matplotlib.pyplot as plt

year = [1950, 1970, 1990, 2010]
pop = [2.519, 3.692, 5.263, 6.972]

plt.clf()
plt.plot(year,pop)
plt.savefig("gp1.png") # or plt.show()
```



What does this line plot show? What does it not show?

→ Four data points connected by a line: world population growth in billions, 1950–2010, from a sample of four years.

→ It does not show anything that happened in between.

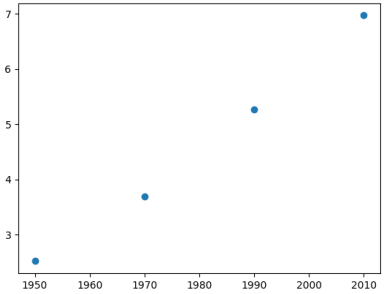
A scatter plot shows the sample data only

```
plt.clf()
plt.scatter(year,pop)
plt.savefig("gp2.png") # or plt.show()
```

Now you have to imagine the line.

When does a line plot make sense?
When does a scatter plot make sense?

→ A line plot makes sense whenever the points are, in fact, connected: a **time series**, a sequence of values ordered by time.
→ A scatter plot is always the basis of a line plot. When the points are not connected, a line plot is meaningless.

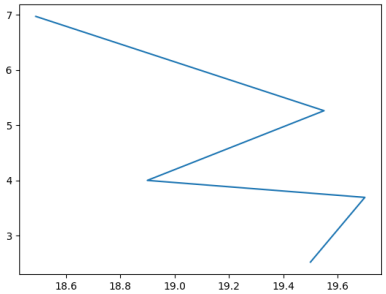


When the points are not connected

```
year = [19.50, 19.70, 18.9, 19.55, 18.49]
pop = [2.519, 3.692, 4, 5.263, 6.972]

plt.clf()
plt.scatter(year,pop)
plt.savefig("gp3.png") # or plt.show()
```

These are just a bunch of random points – the pattern that was present in the population vs. year data is lost.



The GapMinder dataset

→ Switch to editor · code-along · load the data

The "classic" GapMinder dataset, limited to the year 2007: six variables for 142 countries.

```
import pandas as pd

url = "https://raw.githubusercontent.com/birkenkrahe/cosc_1210_fall_26/main/data/gapminder.csv"
df = pd.read_csv(url, index_col=0)

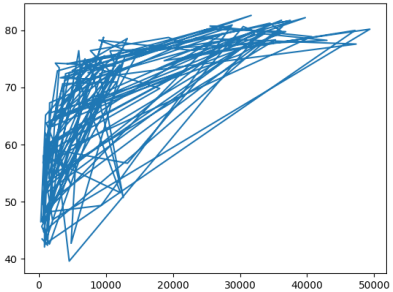
df.info()        # structural information
print(df.head()) # show first few lines
print(df.describe()) # show some statistical properties
```

```
       country  year  population      cont  life_exp       gdp_cap
11 Afghanistan  2007  31889923.0      Asia    43.828    974.580338
23     Albania  2007   3600523.0    Europe    76.423   5937.029526
35     Algeria  2007  33333216.0    Africa    72.301   6223.367465
47      Angola  2007  12420476.0    Africa    42.731   4797.231267
59   Argentina  2007  40301927.0  Americas    75.320  12779.379640
```

Life expectancy vs. GDP per capita

```
plt.clf()
plt.plot(df.gdp_cap,df.life_exp)
plt.savefig("gp4.png")
```

The lines are not helpful here: the points are in no way connected to each other. The life expectancy and GDP of one country is not related to that of another (at least not in a simple way).

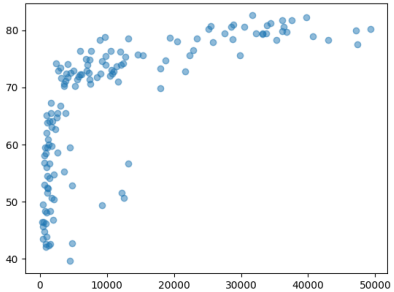


Scatter plot with half transparency

```
plt.clf()
plt.scatter(df.gdp_cap,df.life_exp,alpha=0.5)
plt.savefig("gp5.png")
```

The `alpha` parameter sets the transparency.

The points are spread out across a vast range – the GDP per capita of different countries is very different.

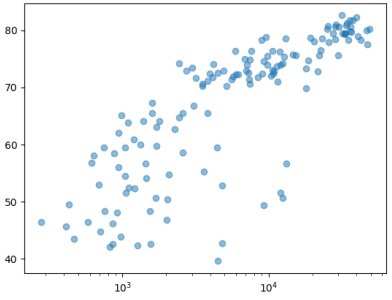


Log scale

A logarithmic transformation of the x-axis pulls the points apart:

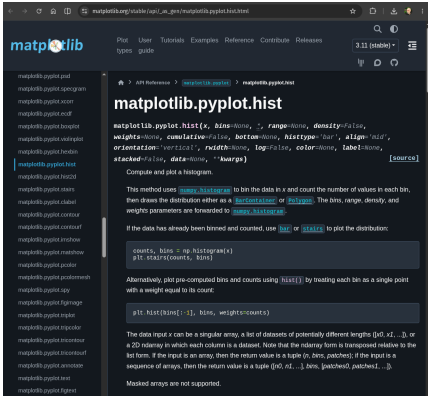
```
plt.clf()
plt.scatter(df.gdp_cap,df.life_exp,alpha=0.5)
plt.xscale('log')
plt.savefig("gp6.png")
```

`plt.xscale('log')` uses base 10 by default: 10,000 becomes 4 because $\log_{10}(10000) = 4$.



Histograms

- Greek roots: *histos* (mast or upright) and *gramma* (drawing or record) – an upright drawing
- `help(plt.hist)` prints the complete docstring, but in the console it is long and unformatted
- The same text, with links and example plots: the online documentation at matplotlib.org
- There is only one positional, non-default argument (x)



bins=None is not "no value"

In the hist function signature, bins=None is not the default value: it's a signal to defer resolution. None triggers a lookup against:

```
import matplotlib
print(matplotlib.rcParams['hist.bins'])
```

```
10
```

rc stands for "runtime configuration". You can use it to look up other stack values:

```
print(matplotlib.rcParams['figure.figsize'])
```

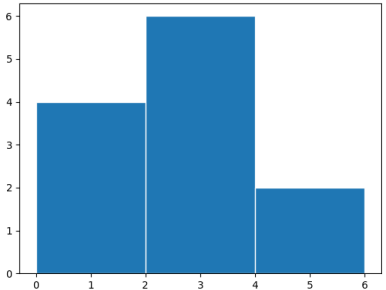
```
[6.4, 4.8]
```

For plots that render through a matplotlib Axes, any keyword defaulting to None is worth suspecting as an rcParams sentinel rather than "no value" (seaborn, pandas, scikit-learn).

A histogram

```
values = [0,0.6,1.4,1.6,2.2,2.5,2.6,3.2,3.5,3.9,4.2,6]
plt.clf()
plt.hist(values,bins=3,edgecolor="white")
plt.savefig("hist.png")
```

Bin	Values	Count
$[0, 2)$	0, 0.6, 1.4, 1.6	4
$[2, 4)$	2.2, 2.5, 2.6, 3.2, 3.5, 3.9	6
$[4, 6]$	4.2, 6	2



Choose the right plot

How do you choose the right plot type?

→ It depends on the type of your data and on what you want to do with the plot.

Histogram

Continuous numeric values (real numbers), e.g. grades – and you're curious about their distribution: all good, all bad, or somewhere in between?

Line plot

Grades recorded over time – are you getting better or worse as time goes by? That's time series data.

Scatter plot

Two continuous numeric variables (pairs of real numbers), e.g. hours studied and the exam score after each study session.

Customization

We're returning to GapMinder for an extended example. Plot customization includes:

- ✓ Plot types (line, bar, hist, scatter, box, pie, etc.)
- ✓ Titles, axis labels
- ✓ Tick marks (position, labels, rotation, formatting)
- ☐ Legends
- ✓ Colors, colormaps
- ☐ Line/marker styles (width, style, marker shape)
- ✓ Annotations (text, arrows)
- ✓ Axis limits and scale (linear/log)
- ✓ Gridlines
- ☐ Figure size, DPI
- ☐ Subplot layout (multiple panels, spacing)
- ☐ Spines (visibility, position)
- ☐ Saving/export format

The basic plot

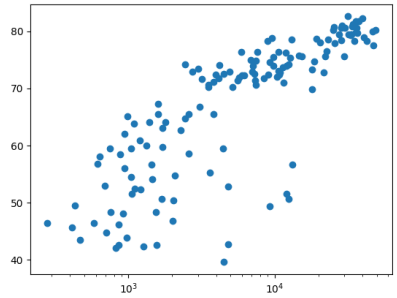
→ **Switch to editor · code-along · GapMinder, refined piece by piece**

Plot life expectancy (in years) as a function of GDP per capita (in USD). Store the variables you work with, for convenience, and look at them.

```
df = df.reset_index(drop=True)

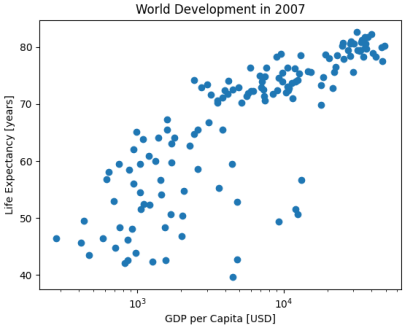
gdp_cap = df['gdp_cap']
life_exp = df['life_exp']
print(gdp_cap)
print(life_exp)

plt.clf()
plt.scatter(gdp_cap,life_exp)
plt.xscale('log')
plt.savefig("labels.png")
```



Axis labels and a title

```
# Strings
xlab = "GDP per Capita [USD]"
ylab = "Life Expectancy [years]"
title = "World Development in 2007"
# Labeled, titled scatter plot, log scale
plt.clf()
plt.scatter(gdp_cap,life_exp)
plt.xscale('log')
plt.xlabel(xlab)
plt.ylabel(ylab)
plt.title(title)
plt.savefig("labels2.png")
```

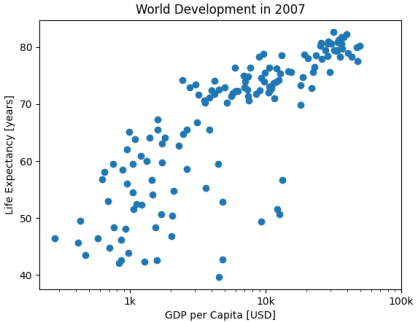


Ticks and tick labels

The GDP is coded in multiples of 10^3 and 10^4 – transform the x-axis tick values and their labels:

```
tick_val = [1000, 10_000, 100_000]
tick_lab = ['1k', '10k', '100k']

# Strings
xlab = "GDP per Capita [USD]"
ylab = "Life Expectancy [years]"
title = "World Development in 2007"
# Labeled, titled scatter plot, log scale
plt.clf()
plt.scatter(gdp_cap,life_exp)
plt.xscale('log')
plt.xlabel(xlab)
plt.ylabel(ylab)
plt.title(title)
plt.xticks(tick_val,tick_lab)
plt.savefig("ticks.png")
```



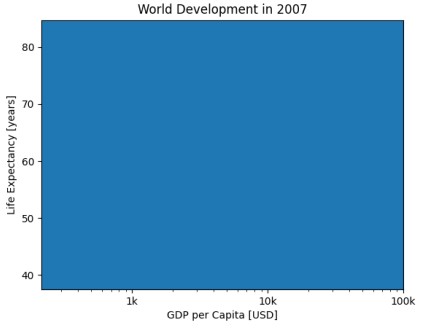
Point sizes: a third dimension

Make the size of each dot correspond to the population of its country. The `s` parameter of `scatter` takes the variable that resizes the dots:

```
pop = df['population']
print(type(pop))
```

```
<class 'pandas.core.series.Series'>
```

```
# ... strings as before
plt.clf()
plt.scatter(gdp_cap,life_exp, s = pop)
plt.xscale('log')
plt.xlabel(xlab)
plt.ylabel(ylab)
plt.title(title)
plt.xticks(tick_val,tick_lab)
plt.savefig("sizes.png")
```



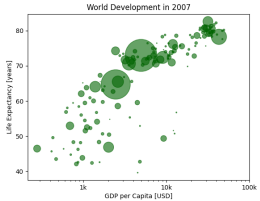
The values are very large: all dots are very large – essentially one large blue dot.

Scale the sizes

Divide pop down to millions, then double each value for greater visibility. Use `alpha` for overlapping points and `color` for visibility:

```
sizes = 2 * pop / 1e6
```

```
# ... strings as before
plt.clf()
plt.scatter(gdp_cap,life_exp, s = sizes, alpha=0.6, color="darkgreen")
plt.xscale('log')
plt.xlabel(xlab)
plt.ylabel(ylab)
plt.title(title)
plt.xticks(tick_val,tick_lab)
plt.savefig("sizes2.png")
```



Colors: one per continent

Color each country by its continent (`cont`). You cannot pass it straight to the `c` parameter of `plt.scatter` – `cont` is categorical:

```
cont = df['cont']
cont.info()
print(cont.unique())
```

```
Series name: cont
142 non-null    object
['Asia' 'Europe' 'Africa' 'Americas' 'Oceania']
```

The color parameter accepts color strings (like `"red"`) or numeric values passed through a colormap. Build a color map as a **dictionary**: keys are the continent names, values the colors.

```
continent_colors = {
    'Asia': 'red',
    'Europe': 'green',
    'Africa': 'blue',
    'Americas': 'yellow',
    'Oceania': 'black'
    }
```

Build the list of colors

Now build the list with the map – one color per country:

```
colors = []
continent = list(cont) # extract continent, store in list
for c in continent:
    colors.append(continent_colors[c])
print([colors[1],colors[-1]])
print(len(colors))
```

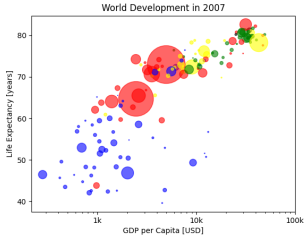
```
['green', 'blue']
142
```

Ready to plot in color

```
# Strings
xlab = "GDP per Capita [USD]"
ylab = "Life Expectancy [years]"
title = "World Development in 2007"
# Labeled, titled scatter plot, log scale
plt.clf()
plt.scatter(gdp_cap,life_exp, s = sizes, alpha=0.6, c=colors)
plt.xscale('log')
plt.xlabel(xlab)
plt.ylabel(ylab)
plt.title(title)

plt.xticks(tick_val,tick_lab)

plt.savefig("colors.png")
```



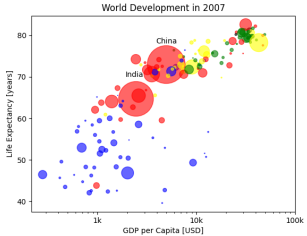
Text

With `plt.text()` you can add text anywhere on your plot. Add the names of the two largest blobs, India and China:

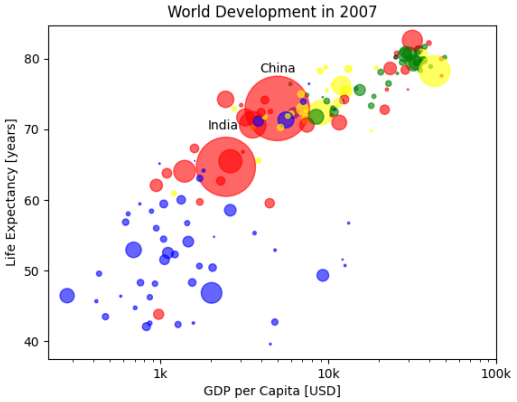
```
# ... strings as before
plt.clf()
plt.scatter(gdp_cap,life_exp, s = sizes, alpha=0.6, c=colors)
plt.xscale('log')
plt.xlabel(xlab)
plt.ylabel(ylab)
plt.title(title)
plt.xticks(tick_val,tick_lab)

plt.text(1900,70, 'India')
plt.text(3900,78, 'China')

plt.savefig("textplot.png")
```



World Development in 2007



Summary

- Data visualization serves two purposes: **explore** your data and **report** your insights. The right plot depends on the data and the story
- `plt.plot(x, y)` draws a **line plot** – only when the points are connected, as in a **time series**. `plt.scatter(x, y)` draws the data points only, for two numeric variables
- `plt.hist(x, bins=3)` draws a **histogram**: the distribution of one variable. `bins=None` means "look up the default" in `rcParams` (10)
- `plt.xscale('log')` pulls apart values that span a vast range (base 10)
- Customize piece by piece: `plt.xlabel`, `plt.ylabel`, `plt.title`, `plt.xticks(values, labels)`, `plt.text(x, y, 'text')`
- `scatter` parameters: `s` sizes (a third dimension), `alpha` transparency, `color` one color, `c` one color per point. A categorical variable needs a dictionary that maps each category to a color
- `plt.clf()` clears the figure; `plt.savefig("file.png")` saves it

Glossary

#	Term	Meaning
1	data visualization	Plots to explore data or report insights
2	line plot	Data points connected by lines (`plt.plot`)
3	scatter plot	Data points only, no lines (`plt.scatter`)
4	time series	A sequence of values ordered by time
5	histogram	Bars showing how one variable is distributed
6	bin	One interval (one bar) of a histogram
7	log scale	Axis on which each step is a factor of 10
8	`rcParams`	Runtime configuration: matplotlib's defaults
9	tick label	The text at a mark on an axis (`plt.xticks`)
10	`alpha`	Transparency, from 0 (invisible) to 1 (solid)
11	categorical variable	Takes one of a fixed set of labels (`cont`)
12	color map	Dictionary that maps each category to a color
13	annotation	Text added to a plot (`plt.text`)